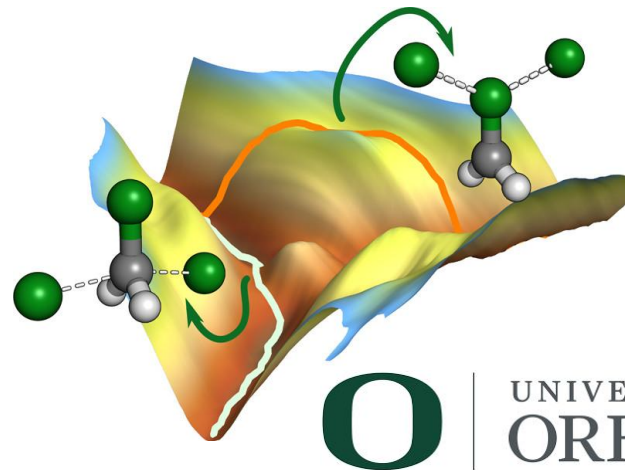
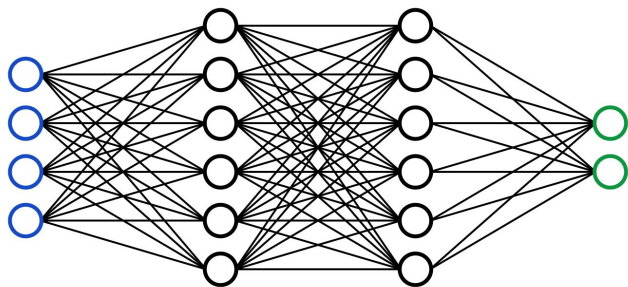


# Machine Learning for Chemistry

Winter 2025

## Topic 2: Challenges in Machine Learning



Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

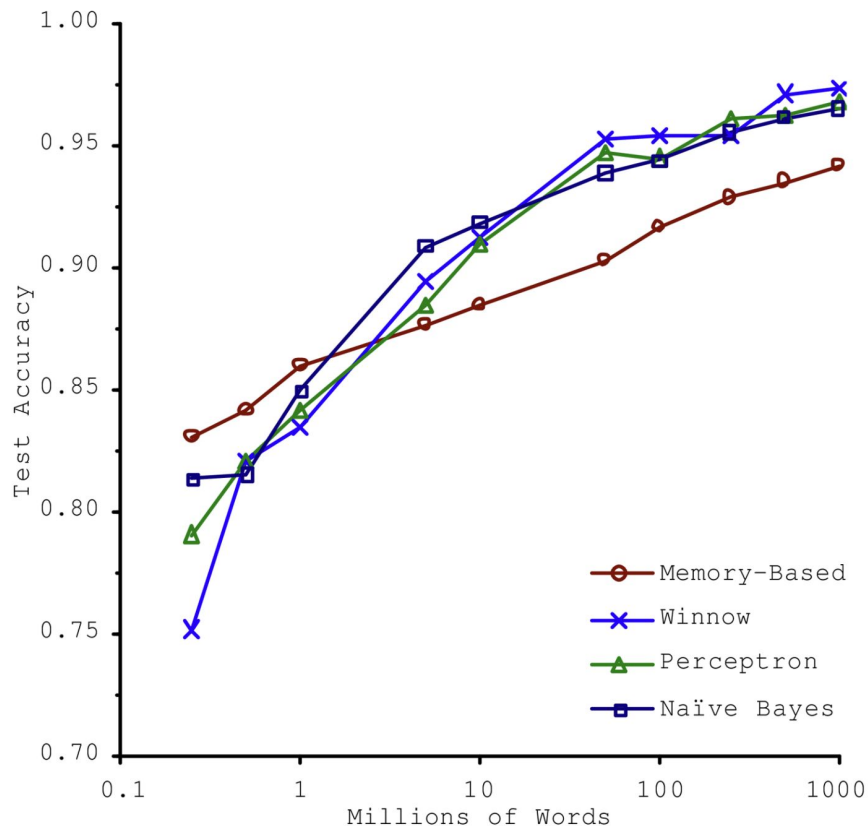
# Overview of Topics

- Amount and Quality of Data
- Generalization
- Bias-Variance Trade-off
- Accuracy vs Interpretability

## References:

1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 1)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 4)
3. **An Introduction to Statistical Learning** by Hastie and Tibshirani (Chapter 2)

# Amount and Quality of Data



Very different Machine Learning algorithms, including fairly simple ones, performed almost identically well on a complex problem of natural language disambiguation once they were given enough data. (Banko and Brill 2001, Microsoft Inc.)

Data often matters more than algorithms for complex problems (Norvig et al. 2009). However, small and medium sized datasets are very common, and it is not always easy or cheap to get extra training data.

# Amount and Quality of Data

## Missing Data

```
>>> df = pd.read_csv(StringIO(csv_data))
```

```
>>> df
```

|   | A    | B    | C    | D   |
|---|------|------|------|-----|
| 0 | 1.0  | 2.0  | 3.0  | 4.0 |
| 1 | 5.0  | 6.0  | NaN  | 8.0 |
| 2 | 10.0 | 11.0 | 12.0 | NaN |

# Amount and Quality of Data

## Missing Data

```
>>> df = pd.read_csv(StringIO(csv_data))
```

```
>>> df
```

|   | A    | B    | C    | D   |
|---|------|------|------|-----|
| 0 | 1.0  | 2.0  | 3.0  | 4.0 |
| 1 | 5.0  | 6.0  | NaN  | 8.0 |
| 2 | 10.0 | 11.0 | 12.0 | NaN |

Delete the rows or columns with missing data

```
>>> df.dropna(axis=0)
```

|   | A   | B   | C   | D   |
|---|-----|-----|-----|-----|
| 0 | 1.0 | 2.0 | 3.0 | 4.0 |

May reduce the dataset size significantly.

```
>>> df.dropna(axis=1)
```

|   | A    | B    |
|---|------|------|
| 0 | 1.0  | 2.0  |
| 1 | 5.0  | 6.0  |
| 2 | 10.0 | 11.0 |

# Amount and Quality of Data

## Missing Data

Delete the rows or columns with missing data

```
>>> df = pd.read_csv(StringIO(csv_data))
```

```
>>> df
```

|   | A    | B    | C    | D   |
|---|------|------|------|-----|
| 0 | 1.0  | 2.0  | 3.0  | 4.0 |
| 1 | 5.0  | 6.0  | NaN  | 8.0 |
| 2 | 10.0 | 11.0 | 12.0 | NaN |

```
>>> df.dropna(axis=0)
```

|   | A   | B   | C   | D   |
|---|-----|-----|-----|-----|
| 0 | 1.0 | 2.0 | 3.0 | 4.0 |

```
>>> df.dropna(axis=1)
```

|   | A    | B    |
|---|------|------|
| 0 | 1.0  | 2.0  |
| 1 | 5.0  | 6.0  |
| 2 | 10.0 | 11.0 |

May reduce the dataset size significantly.

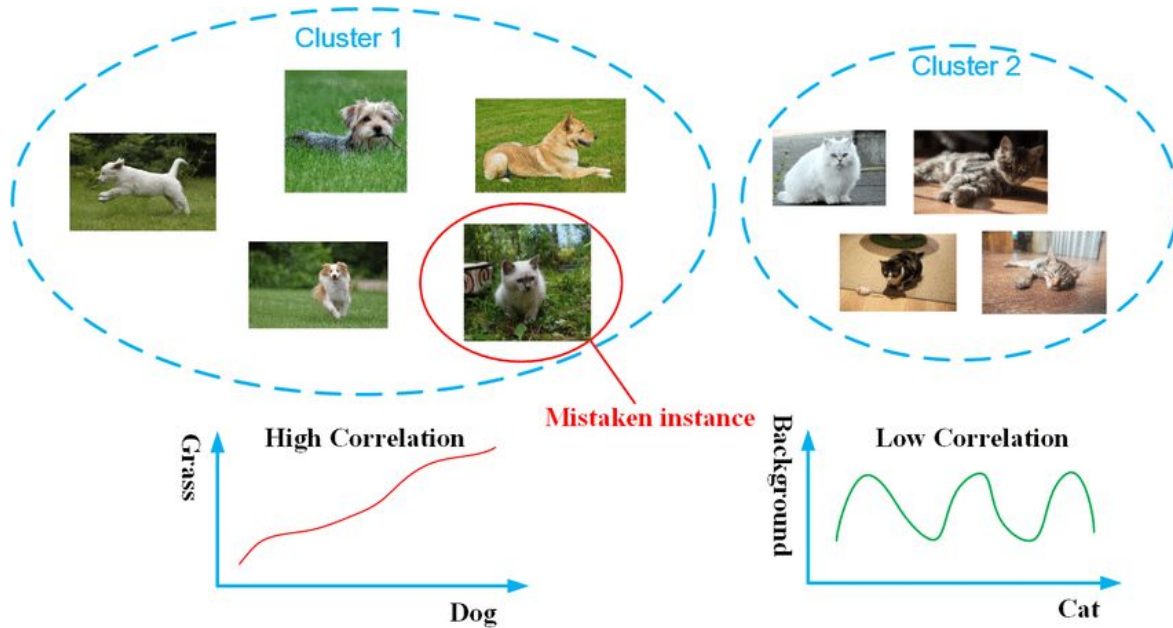
Alternatively, perform interpolation to fill the missing values.

```
>>> from sklearn.impute import SimpleImputer
>>> import numpy as np
>>> imr = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imr = imr.fit(df.values)
>>> imputed_data = imr.transform(df.values)
```

```
>>> imputed_data
array([[ 1.,  2.,  3.,  4.],
       [ 5.,  6.,  7.5,  8.],
       [10., 11., 12.,  6.]])
```

# Amount and Quality of Data

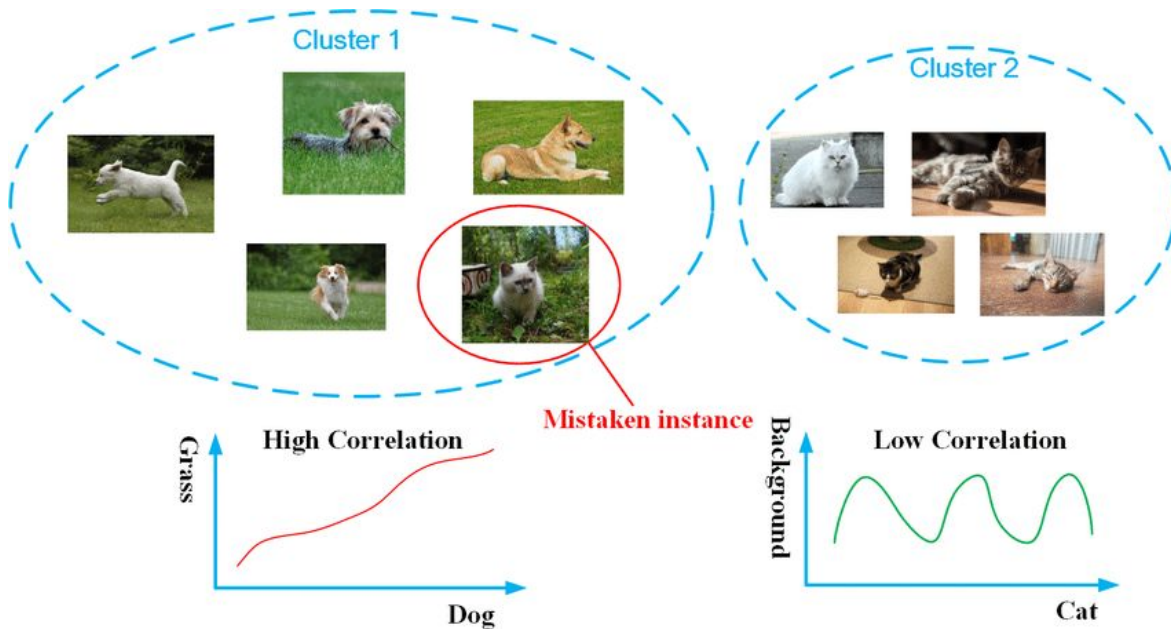
## Correlated Features



1. Remove one of the correlated features.
2. Combine features to reduce dimension.

# Amount and Quality of Data

## Correlated Features



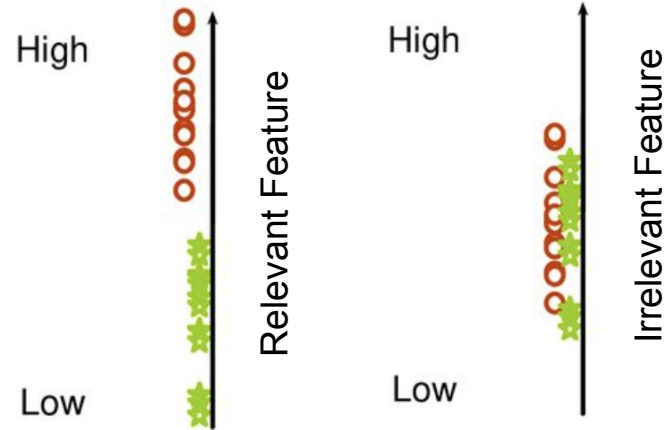
1. Remove one of the correlated features.
2. Combine features to reduce dimension.

Removing one feature may not be great because in combination they can provide important insight, e.g., # Bedroom & square footage of a house.

# Amount and Quality of Data

## Irrelevant Features

1. Discard irrelevant features
2. Alternatively, project the data into more than one features:  
Two irrelevant features can be relevant together



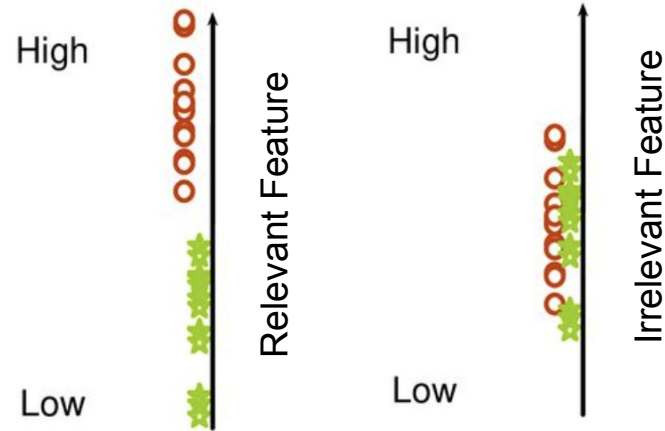
# Amount and Quality of Data

## Irrelevant Features

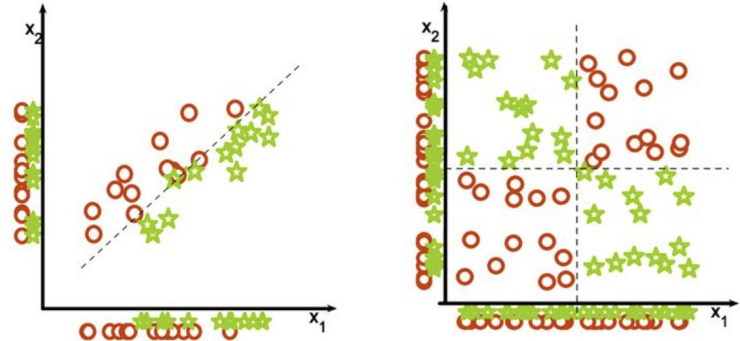
1. Discard irrelevant features
2. Alternatively, project the data into more than one features:  
Two irrelevant features can be relevant together

Combination of two irrelevant features can be relevant!

**Example:** Someone's height or weight by itself does not tell if they are obese. But the combination of height and weight does.

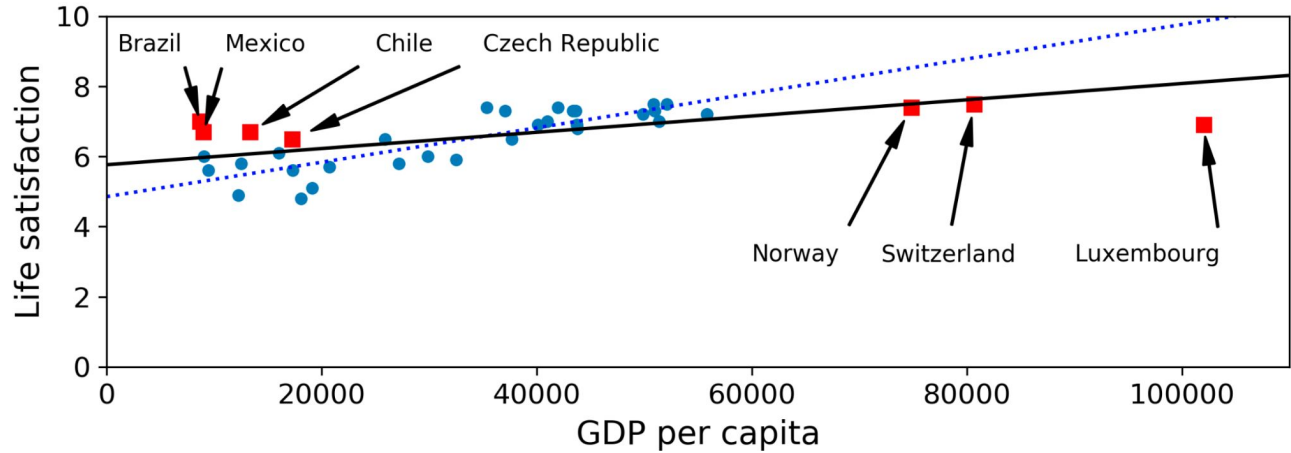


Two irrelevant features can be relevant together



# Generalization

If we train a model on some data and deploy it in real life on new unseen data, it may show poor performance.



A successful machine learning model should be able to perform well on new data (i.e. data that the model has not seen before)

During the training we should split the data into **training**, **validation**, and **test** set.

Train the model just on the **training** data and measure its accuracy on the **validation** and **test** set.

# Training, Validation, and Test Set

**Training Set:** Dataset from which we learn the model parameters.

**Validation Set:** Dataset on which we evaluate the model. If the model performance is not good on Validation Set, we update the model to improve the performance on validation set.

**Test Set:** Dataset on which we test how our model will perform in real life.

Do not use performance on test data to update your model. This will lead to unrealistic model accuracy that will not generalize to “unseen” data

**NEVER TOUCH THE TEST DATA!!!**

# Training, Validation, and Test Set

Reading Textbook



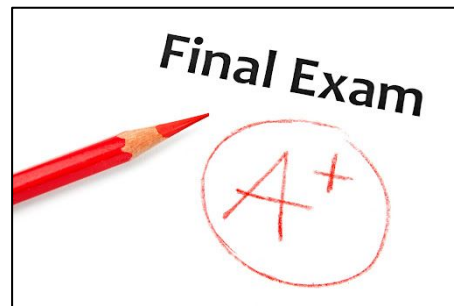
Training Set

Doing Homework



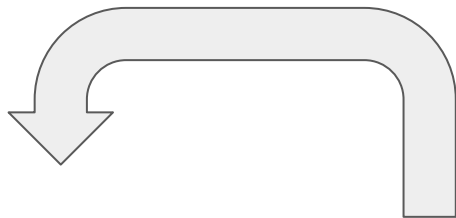
Validation Set

Closed Book Exam



Test Set

# Training, Validation, and Test Set



Reading Textbook



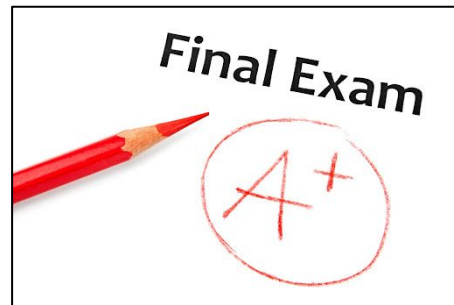
Training Set

Doing Homework



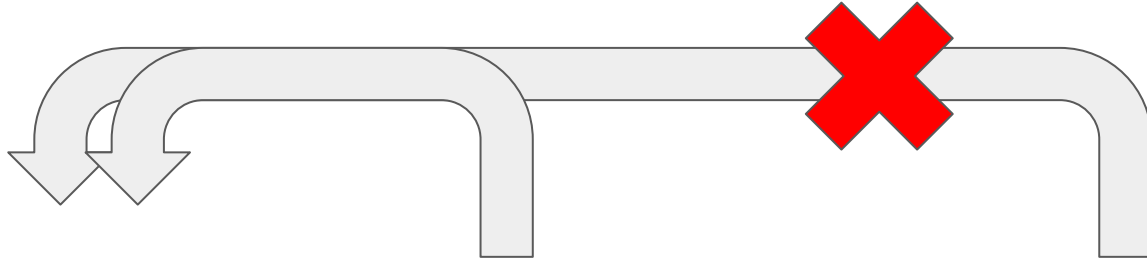
Validation Set

Closed Book Exam



Test Set

# Training, Validation, and Test Set



Reading Textbook



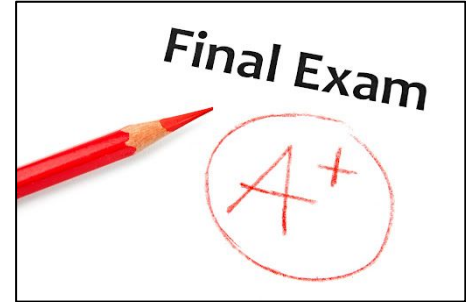
Training Set

Doing Homework



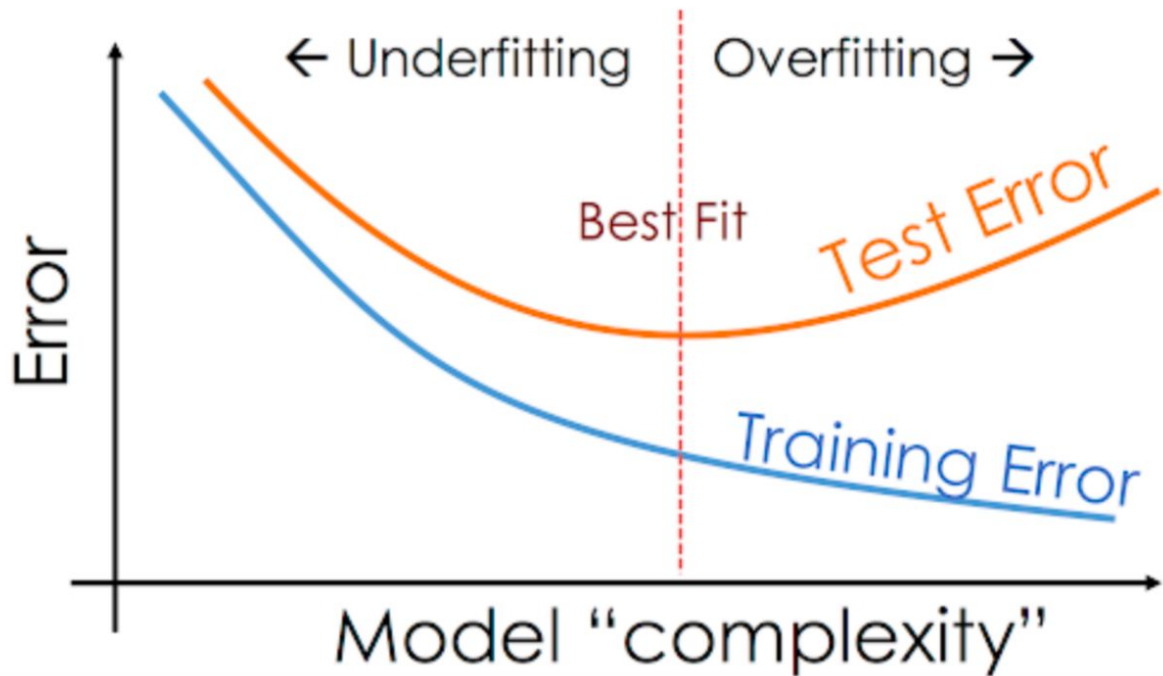
Validation Set

Closed Book Exam



Test Set

# Bias-Variance Trade off



Most complex model is not always the best.

Overtraining can lead to overfitting.

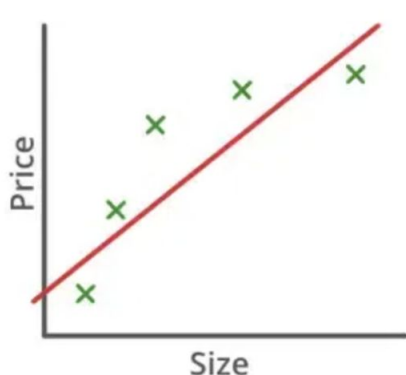
The model memorizes the training set but fails to generalize to new scenarios.

# Bias-Variance Trade off

**Underfitting:** Too simple model cannot learn the logic (**High Bias Low Variance**)

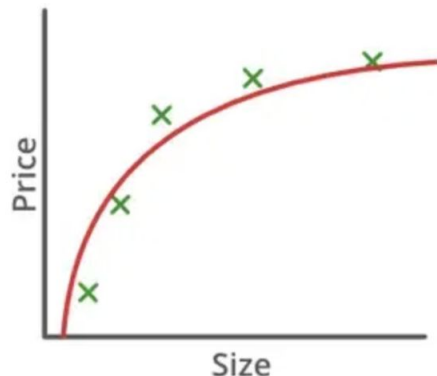
**Overfitting:** Too complex model learns too much noise (**Low Bias High Variance**)

**Good Fitting:** Learns the Logic but ignores the noise.



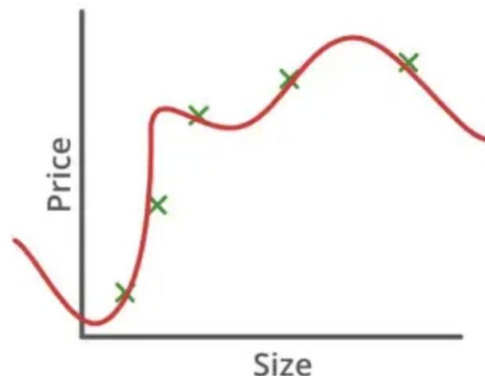
$$\theta_0 + \theta_1 x$$

**High Bias**  
(Underfitting)



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

**Low Bias, Low Variance**  
(Goodfitting)



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

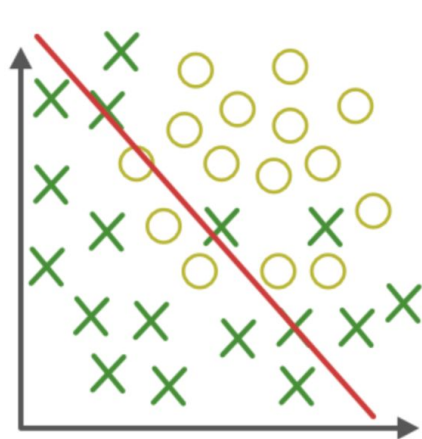
**High Variance**  
(Overfitting)

# Bias-Variance Trade off

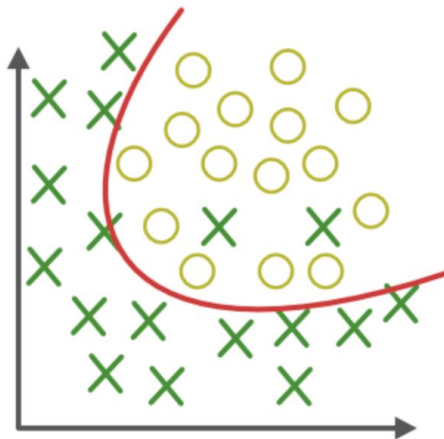
**Underfitting:** Too simple model cannot learn the logic (**High Bias Low Variance**)

**Overfitting:** Too complex model learns too much noise (**Low Bias High Variance**)

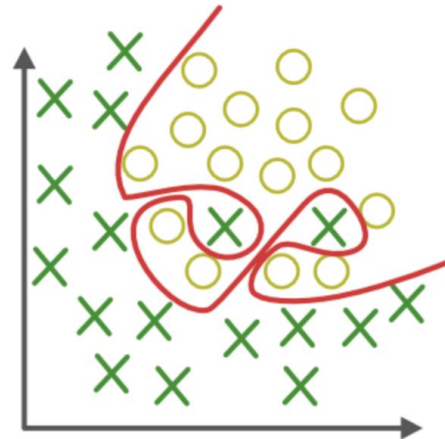
**Good Fitting:** Learns the Logic but ignores the noise.



**Under-fitting**  
(too simple to  
explain the variance)



**Appropriate-fitting**



**Over-fitting**  
(forcefitting--too  
good to be true) 

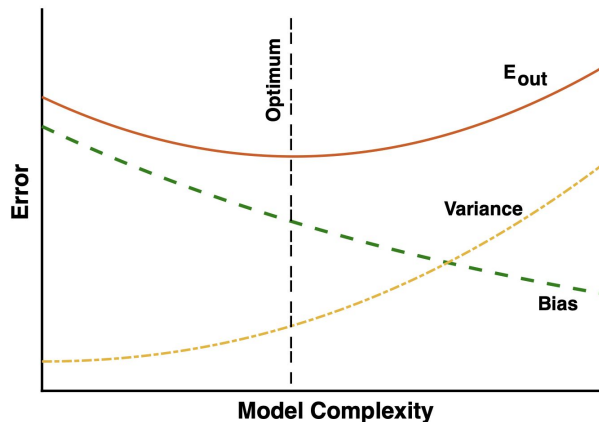
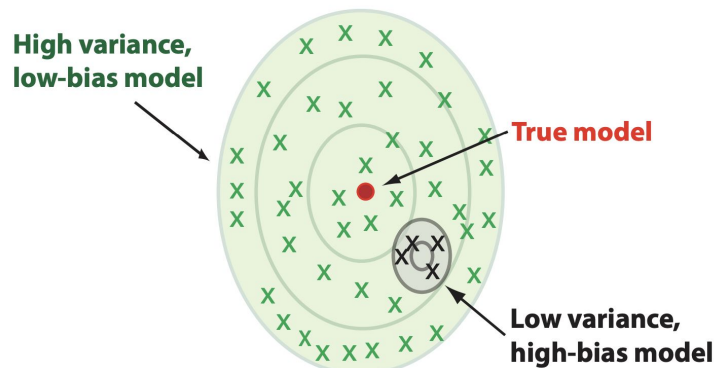
# Bias-Variance Trade off

**High Variance** means the **model can vary/change a lot** if we slightly change the training set. It therefore overfits the training data.

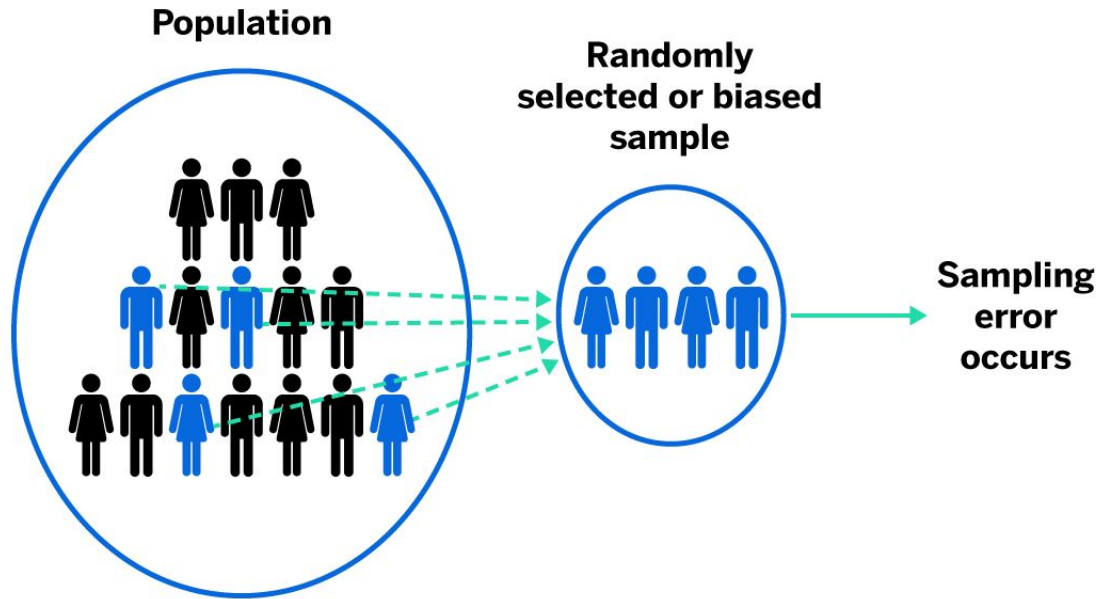
**High Bias** means the model has **very strong inherent assumptions (aka bias)**, e.g. **linear relationship**, and does not learn even when more data is presented.

**Good ML models find a trade-off between these two scenarios.**

Figure: A high-bias, low-variance introduction to Machine Learning for physicists. Mehta et al.

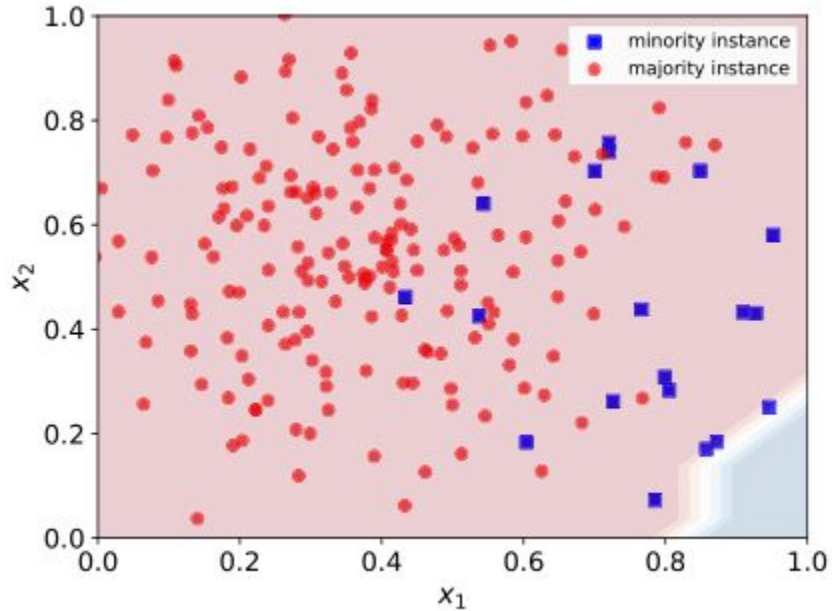


# Bias can come from many places: **Sampling Bias**



<https://ubiai.tools/sampling-bias-in-machine-learning-fresh-update/>

# Bias can come from many places: **Unbalanced Data**



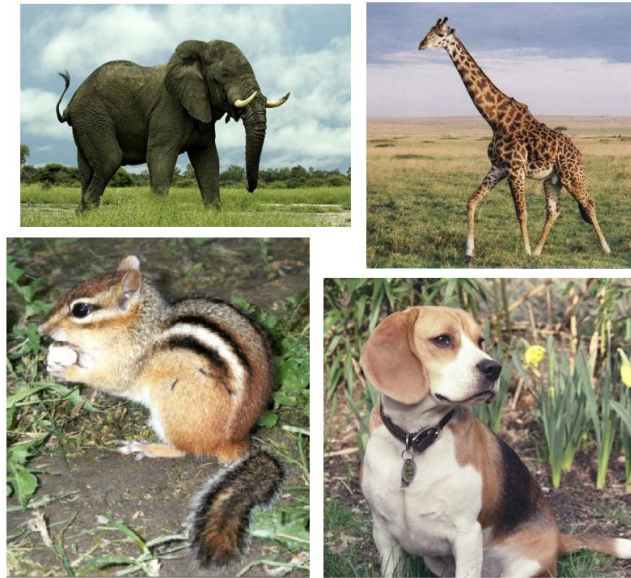
Predict any new data to belong to the majority class:  
High % accuracy but totally meaningless

# Bias can come from many places: **Inductive Bias**

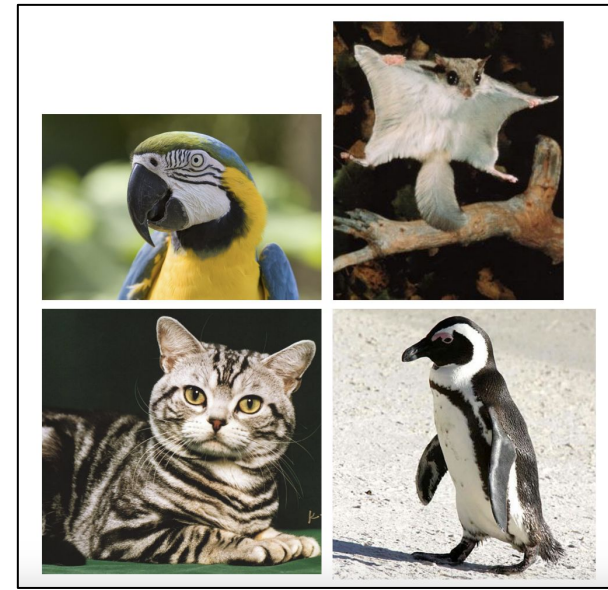
**Class A**



**Class B**



**Test**



# Bias can come from many places: **Inductive Bias**

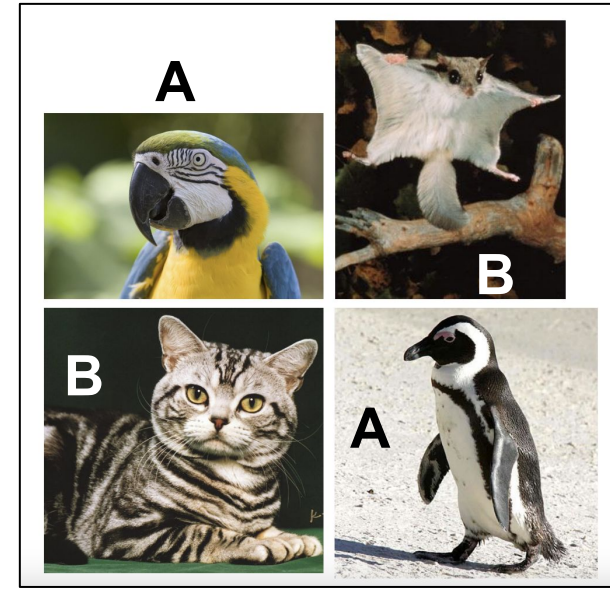
**Class A**



**Class B**



**Test**



How many of you think this is the right answer?

# Bias can come from many places: **Inductive Bias**

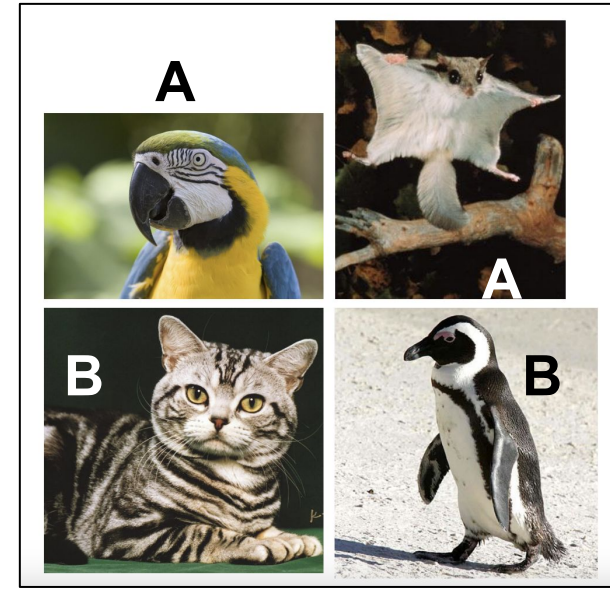
**Class A**



**Class B**



**Test**



And how many for this? Can you think of any other solution?

# Bias can come from many places: **Inductive Bias**

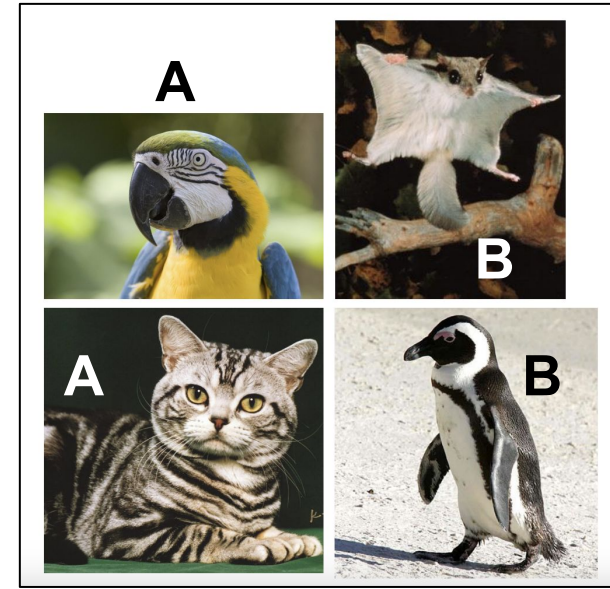
**Class A**



**Class B**



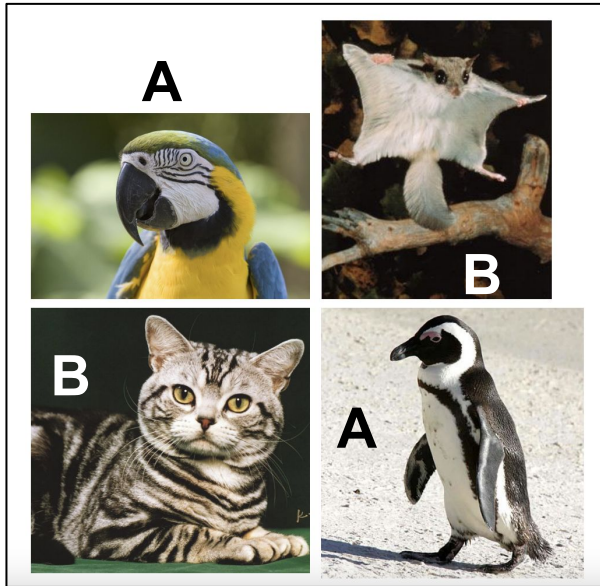
**Test**



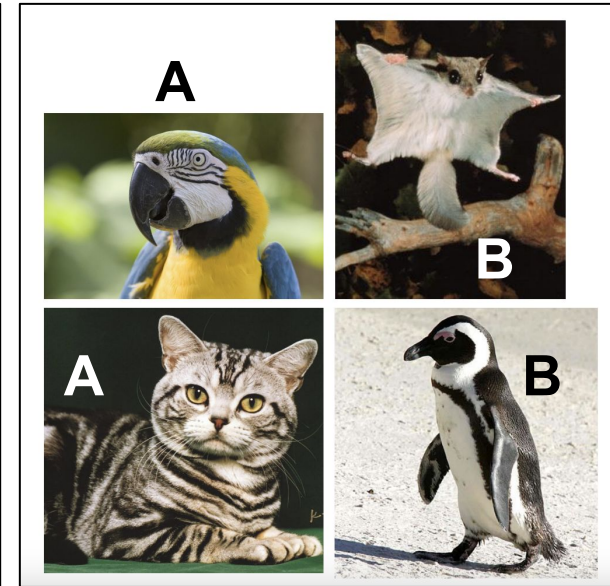
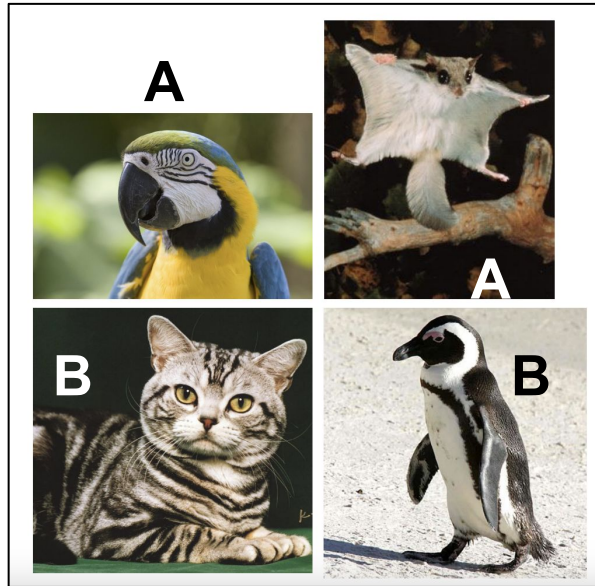
But a Machine Learning model learned this!!! Why?!

# Bias can come from many places: **Inductive Bias**

Bird vs. non bird

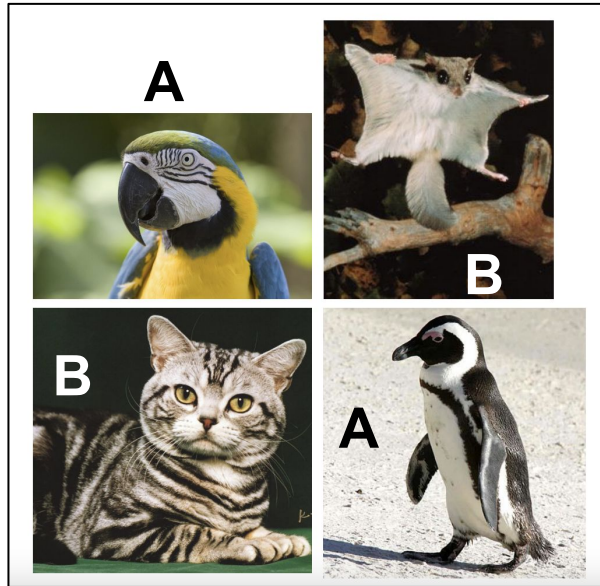


Fly vs. no fly

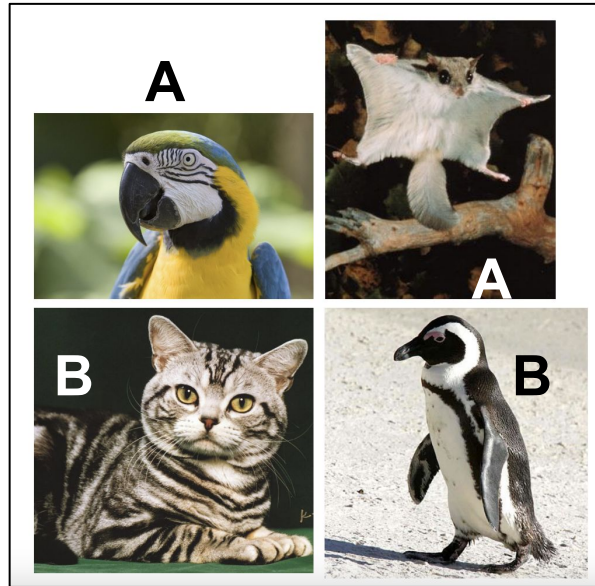


# Bias can come from many places: **Inductive Bias**

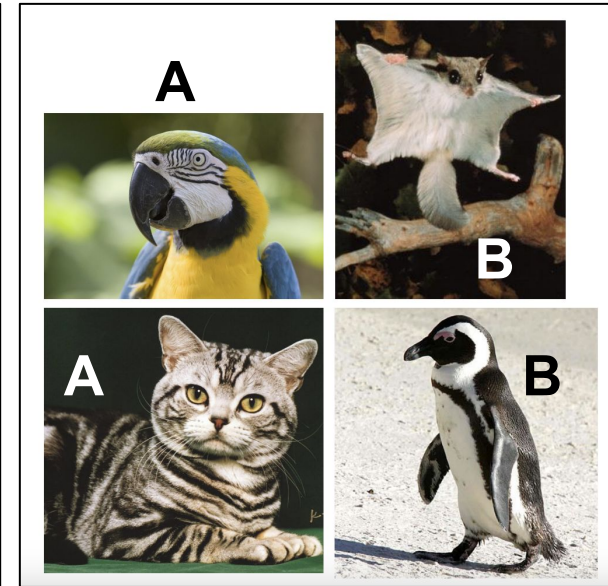
Bird vs. non bird



Fly vs. no fly



Background in focus vs.  
Background off-focus



An ML model trained on pixelated image is more likely to learn the 3rd relationship.

# Bias can come from many places: **Inductive Bias**

**Inductive bias can be quite dangerous!**

**Analysing Chest X-ray Images for Detecting COVID-19:** “Many papers (16 out of 62 studied) used the pneumonia dataset of Kermay et al. as a control group ... consists of paediatric patients aged between one and five. **Developing a model using adult patients with COVID-19 and very young patients with pneumonia is likely to overperform as it is merely detecting children versus adults.**”

[https://www.nature.com/articles/s42256-021-00307-](https://www.nature.com/articles/s42256-021-00307-0)

[0](https://www.nature.com/articles/s42256-021-00307-0)



(a) Normal



(b) Bacterial Pneumonia

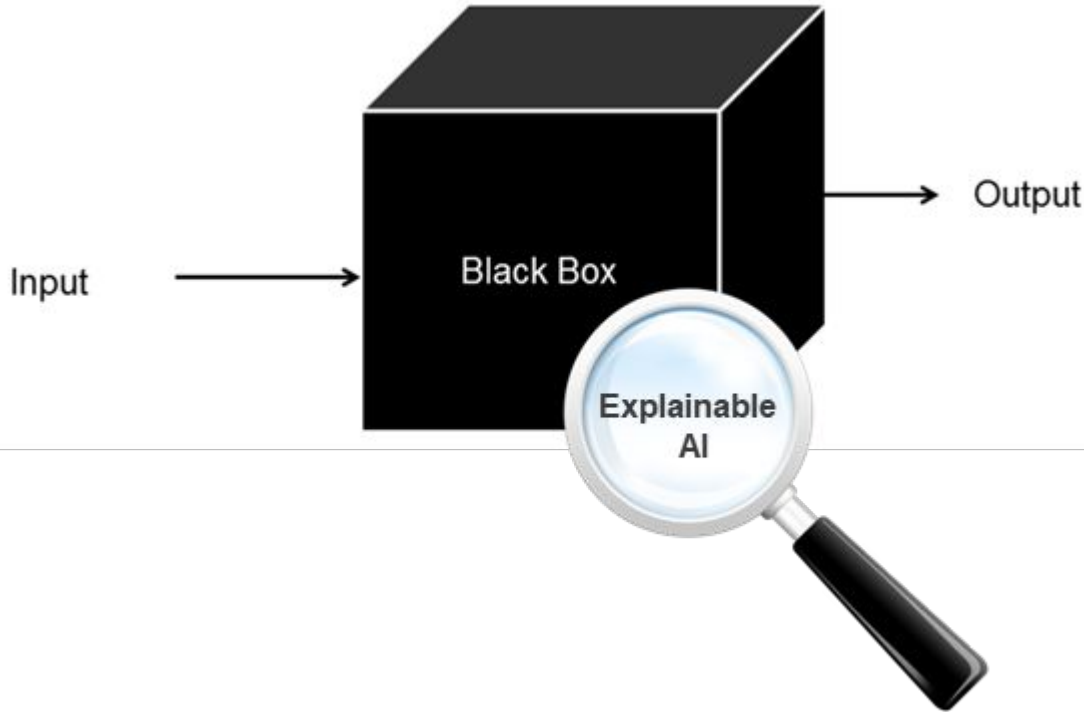


(c) Viral Pneumonia



(d) COVID-19 Pneumonia

# Accuracy vs Interpretability

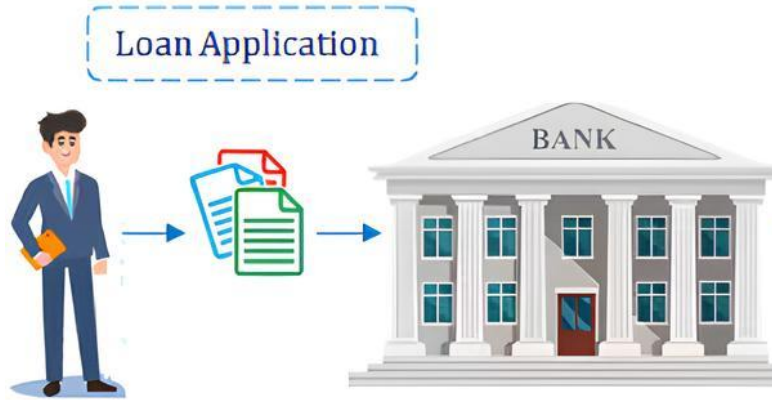


Due to seemingly uncontrollable biases it is even more important to explain what actually an ML model is learning.

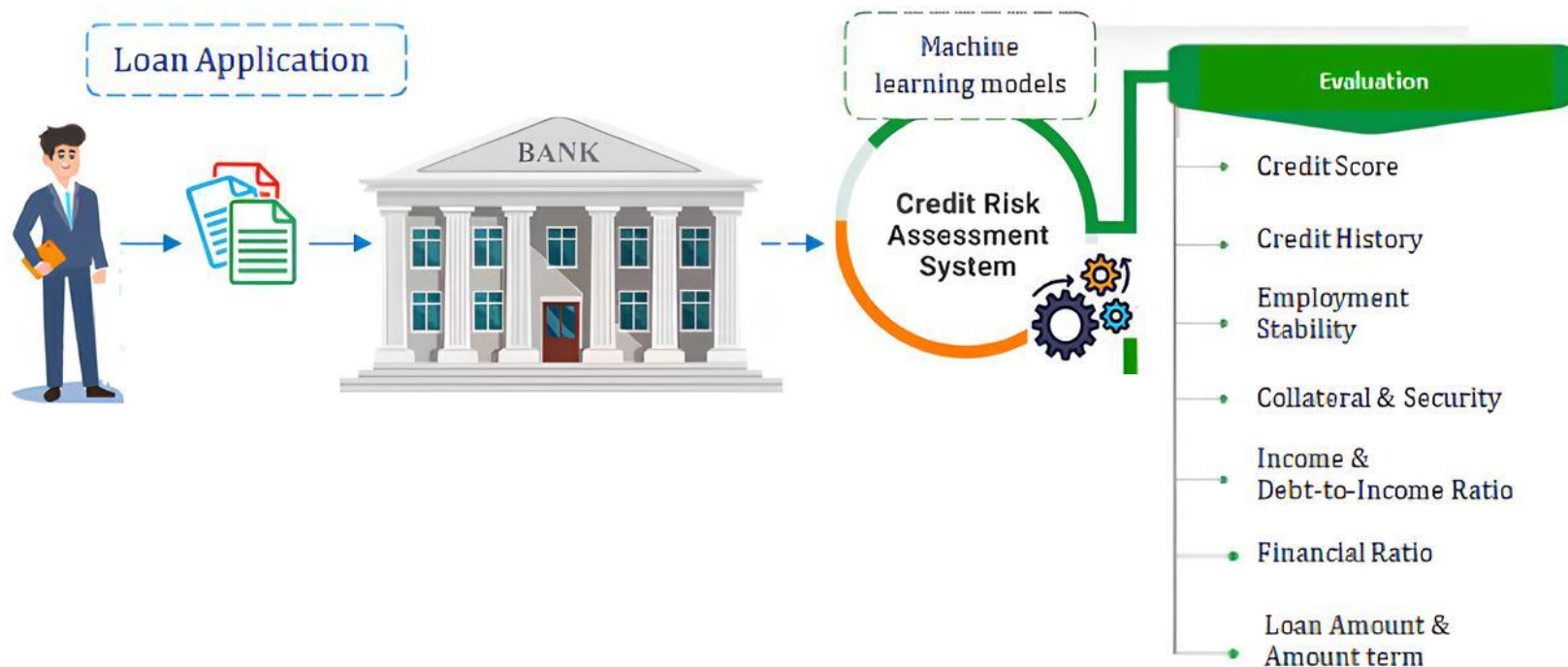
Sometime simpler models with better interpretability is more useful than highly accurate but complex models that are totally black-box.

<https://pub.aimind.so/demystifying-the-black-box-interpretable-machine-learning-6388bef4aeca>

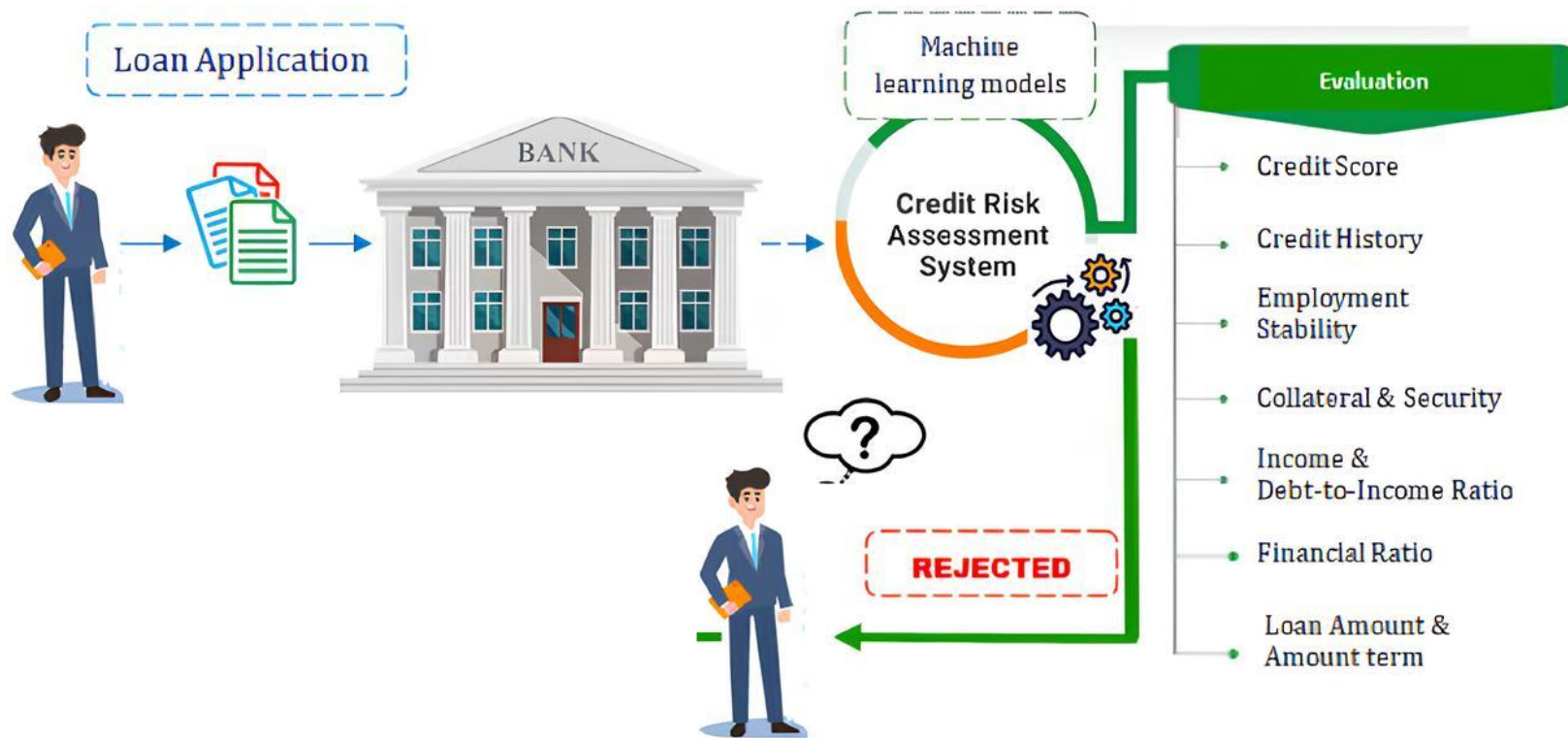
# Accuracy vs Interpretability



# Accuracy vs Interpretability



# Accuracy vs Interpretability



# Accuracy vs Interpretability

